By TOUCH DEN · November 11, 2026

Binary, Hexadecimal, and Bits

Video 1 · Bits, Bytes, and Binary

How Computers Work · Section 2

In this video, you will learn:

- What bits and bytes are
- How to count in binary
- Converting between binary and decimal

Bits and bytes

At the bottom, everything is switches: **off = 0**, **on = 1**. One switch is a **bit**.

One bit says little (2 values). So we group them: **8 bits = 1 byte**.

Term	Meaning
Bit	One switch: 0 or 1
Byte	8 bits; **256** values (0–255)
Nibble	4 bits (half a byte)

Eight bits give 2^8 = **256** patterns, `00000000` to `11111111`. Remember **256** and **255**.

Counting in binary

Decimal (base 10): columns are ×10, ones/tens/hundreds. Binary (base 2): columns **double**, 1, 2, 4, 8, 16, 32, 64, 128.

To read binary, add the column values where there's a 1:

Binary	128	64	32	16	8	4	2	1	=
`0000 1101`					1	1		1	**13**
`1111 1111`	1	1	1	1	1	1	1	1	**255**

Same counting you know, with two digits, because a switch has two states.

Converting, and why you don’t memorize it

- **Binary → decimal:** add the column values where there’s a 1. `0000 1101` = 8 + 4 + 1 = 13.
- **Decimal → binary:** take the biggest power of two that fits, subtract, repeat.

A tool checks you (shown as it ran):

```
$ python3 -c 'print(bin(13), 13); print(int("1010", 2))'
0b1101 13
10
```

`0b` means ”binary”. **Understand the columns; let a tool do the arithmetic.**

Recap: bits, bytes, binary

- A **bit** is one switch; **8 bits = 1 byte** = 256 values (0–255).
- Binary columns **double**: 1, 2, 4, 8, …; read it by adding columns with a 1.
- Don’t memorize conversions, understand the columns; tools convert.

Next video: hex, bitwise operations, and reading a hex dump.

Binary, Hexadecimal, and Bits

Video 2 · Hex, Bitwise, and Hex Dumps

How Computers Work · Section 2

In this video, you will learn:

- What hexadecimal is, and why tools use it
- Bitwise AND, OR, XOR, NOT
- How to read a hex dump

Hexadecimal

Binary is correct but long (`11111111` = one byte). Tools use **hex** (base 16): digits 0–9, then **A–F** for 10–15.

Why 16? One hex digit = **4 bits**; two hex digits = **one byte**. A short, exact way to write bytes.

Binary	Hex	Decimal
`1010`	A	10
`1111`	F	15
`1111 1111`	FF	255

One byte is always **two hex digits**: `00` to `FF`. Written `0xFF`.

Bitwise: AND, OR, XOR, NOT

Computers work on bits directly, one position at a time. Everywhere in permissions, network masks, and simple ciphers.

- **AND** → 1 if **both** are 1. **OR** → 1 if **either**. **XOR** → 1 if **different**. **NOT** → flip.

```
$ python3 -c 'a,b=0b1100,0b1010
print(f"{a:04b} XOR {b:04b} = {a^b:04b}")'
1100 XOR 1010 = 0110
```

XOR matters most: apply it twice and you get the original back, the heart of simple ciphers (Cryptography course).

Reading a hex dump

A **hex dump** shows raw bytes in hex, files, memory, packets. Three columns:

```
$ printf 'Hi!' | xxd
00000000: 4869 21                                  Hi!
```

- `00000000` = the **offset** (how far in, in hex).
- `4869 21` = the **bytes**, in hex: `48`, `69`, `21`.
- `Hi!` = those bytes as **text**.

So `H` = `0x48`, `i` = `0x69`, `!` = `0x21`. (Why? Next lesson.)

Same bits, different meaning

One more idea: a byte's bits are fixed, but what number they *mean* depends on how it's read.

- `1111 1111` = **255** read as unsigned, or **-1** read as signed (two's complement).

The bits are identical; only the interpretation differs.

This is the lesson-1 idea again, and the root of **integer and sign bugs** you'll meet in Binary Exploitation.

Recap: hex, bitwise, dumps

- **Hex** is a short, exact way to write bytes: two digits per byte, `0x` prefix.
- **AND / OR / XOR / NOT** work on bits; XOR underlies simple ciphers.
- A **hex dump** = offset · hex bytes · text; the everyday view of raw data.
- Same bits can mean different numbers (255 vs −1), interpretation is everything.

Next lesson: how text becomes numbers, ASCII, Unicode, and Base64.